

Description de workflow Git

ROUXEL Martin
VO Thanh-Thang
MAKNI Oussama

5 novembre 2025



Partenaires



.....

Ce document a pour objectif de formaliser le workflow Git à utiliser par les membres de l'équipe Sonic durant la réalisation du projet. Il offre une courte introduction à Git puis détaille le workflow de l'équipe. Des commandes standard pour sa mise en pratique seront également fournies et leur utilisation est encouragée.

Table des matières

1	Introduction à Git	3
1.1	Pourquoi utiliser Git ?	3
1.2	Fonctionnement de base	3
1.3	Bonnes pratiques	3
2	Workflow Git	4
2.1	Introduction	4
2.2	Choix du workflow	4
2.3	Spécifications	5
2.3.1	Création des repositories	5
2.3.2	Mise à jour du repository public	5
2.3.3	Création d'une branche de feature	6
2.3.4	Création d'une branche de développement personnelle	6
2.3.5	Cycle de travail	6
3	Setup	7
3.1	Initialisation du dépôt local	7
3.1.1	Clonage du dépôt privé	7
3.1.2	Ajout du remote public	7
3.1.3	Récupération des branches distantes	8
3.1.4	Mise à jour de la branche principale locale	8
3.2	Commandes essentielles par étape de développement	8

1 Introduction à Git

Git est un système de gestion de versions. Il permet à plusieurs développeurs de travailler simultanément sur un même projet sans écraser les modifications des autres. Chaque utilisateur possède une copie complète du dépôt, ce qui rend les opérations locales rapides et sûres.

1.1 Pourquoi utiliser Git ?

- **Historique complet** : chaque modification du code est enregistrée, permettant de revenir facilement à une version antérieure.
- **Travail collaboratif** : plusieurs développeurs peuvent travailler sur différentes branches puis fusionner leurs changements.
- **Sécurité et traçabilité** : chaque commit est signé et horodaté, garantissant l'intégrité du code et facilitant le suivi des contributions.

1.2 Fonctionnement de base

Le dépôt (*repository*) contient tout l'historique du projet. Chaque développeur clone ce dépôt pour obtenir une copie locale :

```
git clone <url>
```

Les fichiers modifiés doivent ensuite être suivis et validés :

```
.....% Vérifie les changements
git status
git add <fichier> % Ajoute le fichier à l'index
git commit -m "Message descriptif" % Enregistre les modifications
.....
```

Pour partager les changements avec les autres :

```
.....% Envoie les commits vers le dépôt distant
git push
git pull % Récupère les mises à jour du dépôt distant
.....
```

Les branches permettent de développer de nouvelles fonctionnalités sans perturber la version principale :

```
.....% Crée une nouvelle branche
git branch <nom>
git switch <nom> % Bascule sur la branche
git merge <nom> % Fusionne la branche dans la branche courante
.....
```

1.3 Bonnes pratiques

- Commit souvent, avec des messages clairs.
- Travailler sur des branches dédiées aux fonctionnalités.
- Effectuer régulièrement un `git pull` avant de pousser vos changements.

Git constitue ainsi un outil essentiel pour assurer la cohérence, la collaboration et la pérennité d'un projet logiciel (et dans notre cas matériel).

2 Workflow Git

2.1 Introduction

Un **workflow Git** désigne l'organisation et les règles de collaboration autour de l'utilisation de Git dans un projet. Il définit la manière dont les développeurs créent, modifient, valident et intègrent le code au dépôt principal.

Un workflow précise notamment :

- comment les branches sont utilisées (par exemple, une branche **main** stable et des branches de développement ou de fonctionnalités) ;
- à quel moment et comment les changements sont fusionnés (**merge** ou **rebase**) ;
- qui révise ou valide le code avant son intégration ;
- le cycle type d'une contribution (création de branche, développement, test, fusion).

Le choix d'un workflow dépend de la taille de l'équipe et du type de projet. Parmi les modèles courants, on trouve :

- le **Git Flow**, avec des branches dédiées aux fonctionnalités, aux versions et aux correctifs ;
- le **Feature Branch Workflow**, plus simple, où chaque nouvelle fonctionnalité a sa propre branche ;
- le **Forking Workflow**, souvent utilisé pour les projets open source, où chaque contributeur travaille sur sa propre copie du dépôt.

2.2 Choix du workflow

Dans notre cas, nous utiliserons un workflow proche du **Feature Branch Workflow**.

Le projet dépend de trois *repositories* différents :

1. **cva6-softcore-contest** → la version des organisateurs
2. **cva6-softcore-contest-sonic** → le fork public de rendu du projet
3. **cva6-softcore-contest-sonic-private** → notre version de travail privée

cva6-softcore-contest-sonic peut être synchronisé si besoin avec le *repo* des organisateurs depuis GitHub (sync fork). Le *repo* privé, **cva6-softcore-contest-sonic-private**, est une copie conforme du fork public au démarrage.

L'idée générale de notre workflow est que les développeurs créent leurs branches de développement à partir des branches de *feature*, elles-mêmes dérivées de la branche principale du *repo* public. Cela permet à chaque contributeur de travailler sur une base de code stable et toujours à jour avec la version officielle.

À la fin du concours, l'ensemble des modifications apportées au *repo* privé sera fusionné dans le *repo* public.

2.3 Spécifications

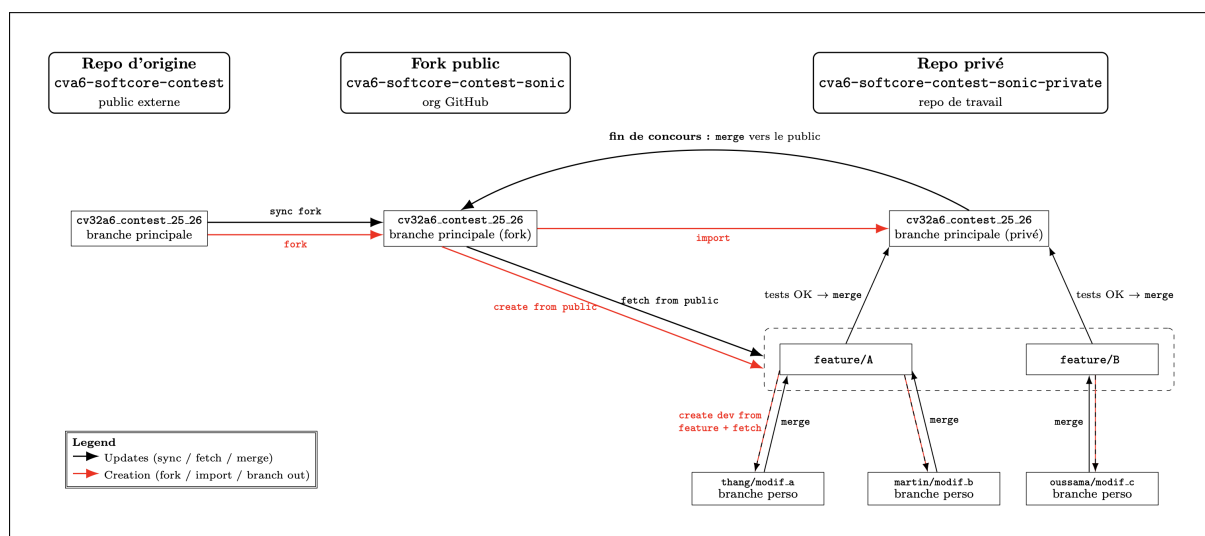


FIGURE 1 – Représentation du workflow

Les spécifications du workflow sont représentées graphiquement sur la figure 1 et sont détaillées de la manière suivante :

2.3.1 Création des repositories

`cva6-softcore-contest-sonic` est un fork du *repo* des organisateurs ; il sera régulièrement synchronisé avec ce dernier en cas de modifications.

`cva6-softcore-contest-sonic-private` est un import GitHub de la version publique. Il est identique au premier lors de sa création et sera mis à jour régulièrement par les développeurs au cours du projet.

2.3.2 Mise à jour du repository public

Avant toute création de branche, il est nécessaire de s'assurer que la copie locale du repository public est synchronisée avec la version la plus récente :

```
.....
git fetch public
git switch cv32a6.contest_25_26
git pull public cv32a6.contest_25_26
.....
```

Ici, `public` désigne la *remote* du fork public `cva6-softcore-contest-sonic`. Cette étape garantit que la branche principale locale est alignée avec celle du repository public.

2.3.3 Création d'une branche de feature

Les branches de *feature* sont créées directement à partir de la branche principale du *repo public*. Chaque fonctionnalité ou groupe de modifications majeures doit disposer de sa propre branche :

```
.....  
git switch -c feature/<nom_feature> public/cv32a6.contest_25_26  
.....
```

Cette approche garantit que les nouvelles fonctionnalités reposent toujours sur la version la plus récente du code validé publiquement.

2.3.4 Création d'une branche de développement personnelle

Chaque développeur travaille sur une branche dérivée de la branche de *feature* qui lui est attribuée. La convention de nommage adoptée est : <prenom>/<modif>.

```
.....  
git switch -c <prenom>/<modif> feature/<nom_feature>  
.....
```

Exemple :

```
.....  
git fetch public  
git switch feature/A  
git pull private feature/A  
  
git switch -c martin/a feature/A  
.....
```

Le développeur effectue ensuite ses commits et pousse régulièrement son travail vers le repository privé.

2.3.5 Cycle de travail

Le cycle de développement s'organise comme suit :

1. Le développeur travaille sur sa branche personnelle et pousse son travail sur le repository privé.
2. Avant chaque push important, il effectue un `git pull` de la branche principale publique pour s'assurer de la compatibilité et tester la non-régression.
3. Une fois la modification testée et validée, il propose une fusion (*merge*) vers la branche `feature/<nom_feature>`.
4. Lorsque la feature est complète et approuvée collectivement, elle est fusionnée dans la branche principale privée `cv32a6.contest_25_26`.

3 Setup

Cette section décrit les étapes nécessaires pour initialiser un dépôt Git local compatible avec le workflow décrit précédemment. Chaque développeur doit disposer d'un dépôt local configuré avec deux remotes distincts :

- **private** : remote principal pointant vers `cva6-softcore-contest-sonic-private` (dépôt de travail de l'équipe) ;
- **public** : remote secondaire pointant vers `cva6-softcore-contest-sonic` (fork public synchronisé avec les organisateurs).

3.1 Initialisation du dépôt local

3.1.1 Clonage du dépôt privé

Lien : <https://github.com/sonic-cva6-team/cva6-softcore-contest-sonic-private>

Chaque développeur commence par cloner le dépôt privé de l'équipe :

```
.....  
git clone <url-privé>  
cd cva6-softcore-contest-sonic-private  
git submodule update --init --recursive  
.....
```

Par défaut, le remote créé s'appelle **origin** et pointe vers le dépôt privé. Pour suivre la convention utilisée dans ce document, on renomme ce remote en **private** :

```
.....  
git remote rename origin private  
.....
```

3.1.2 Ajout du remote public

Lien : <https://github.com/sonic-cva6-team/cva6-softcore-contest-sonic>

On ajoute ensuite le remote **public**, qui pointe vers le fork de rendu :

```
.....  
git remote add public <url-public>  
.....
```

On peut vérifier la configuration des remotes avec :

```
.....  
git remote -v  
.....
```

On doit obtenir quelque chose de la forme :

```
> git remote -v  
private https://github.com/sonic-cva6-team/cva6-softcore-contest-sonic-private (fetch)  
private https://github.com/sonic-cva6-team/cva6-softcore-contest-sonic-private (push)  
public https://github.com/sonic-cva6-team/cva6-softcore-contest-sonic (fetch)  
public https://github.com/sonic-cva6-team/cva6-softcore-contest-sonic (push)
```

3.1.3 Récupération des branches distantes

Une fois les remotes configurés, on récupère l'ensemble des références :

```
.....  
git fetch private  
git fetch public  
.....
```

La branche principale utilisée pour le développement est :

```
.....  
cv32a6.contest_25_26  
.....
```

On peut se placer dessus à partir du dépôt privé ou du dépôt public :

```
.....  
git switch cv32a6.contest_25_26  
# ou explicitement depuis le remote public (non recommandé)  
git switch -c cv32a6.contest_25_26 public/cv32a6.contest_25_26  
.....
```

3.1.4 Mise à jour de la branche principale locale

Avant de créer une nouvelle branche de *feature*, on s'assure que la branche locale principale est à jour avec le dépôt public :

```
.....  
git fetch public  
git switch cv32a6.contest_25_26  
git pull public cv32a6.contest_25_26  
.....
```

Cette branche locale servira ensuite de base à la création des branches de *feature*.

3.2 Commandes essentielles par étape de développement

Cette sous-section récapitule les commandes clés à utiliser à chaque étape du cycle de développement.

1. Mettre à jour la base de travail depuis le public

```
.....  
git fetch public  
git switch cv32a6.contest_25_26  
git pull public cv32a6.contest_25_26  
.....
```

2. Créer une nouvelle branche de feature

```
.....  
git switch -c feature/<nom_feature> public/cv32a6.contest_25_26  
git push private feature/<nom_feature>  
.....
```


3. Créer une branche de développement personnelle

```
.....  
git switch feature/<nom_feature>  
git pull private feature/<nom_feature>    # s'assurer d'être à jour  
git switch -c <prenom>/<modif> feature/<nom_feature>  
git push private <prenom>/<modif>  
.....
```

4. Travail quotidien sur la branche de développement

```
.....  
# Vérifier l'état  
git status  
  
# Ajouter les fichiers modifiés  
git add <fichier1> <fichier2>  
  
# Créer un commit  
git commit -m "Description courte et précise"  
  
# Mettre à jour la branche avec la feature (si besoin)  
git pull --rebase private feature/<nom_feature>  
  
# Pousser le travail sur le dépôt privé  
git push private <prenom>/<modif>  
.....
```

5. Préparer la fusion vers la branche de feature

Avant de proposer la fusion, on s'assure que la branche personnelle est à jour et qu'il n'y a pas de régressions :

```
.....  
git fetch private  
git switch <prenom>/<modif>  
git pull --rebase private feature/<nom_feature>  
# exécuter les tests locaux ici  
.....
```

Une fois la branche validée, la fusion vers `feature/<nom_feature>` se fait via une merge request / pull request sur le dépôt privé.

